



Programação Avançada 2025-26

[18] Padrões de Software | Command

Bruno Silva, Patrícia Macedo

Sumário



- Padrão **Command**
 - Enquadramento
 - Motivação
 - Solução Proposta (pelo padrão)
 - Aplicações genéricas
 - Exemplo de Aplicação

Command | Enquadramento

- O padrão **command** é um padrão de desenho *comportamental*.



Padrão Comportamental

Estes padrões incidem sobre algoritmos e na atribuição/divisão de responsabilidades entre classes.

Command | Problema

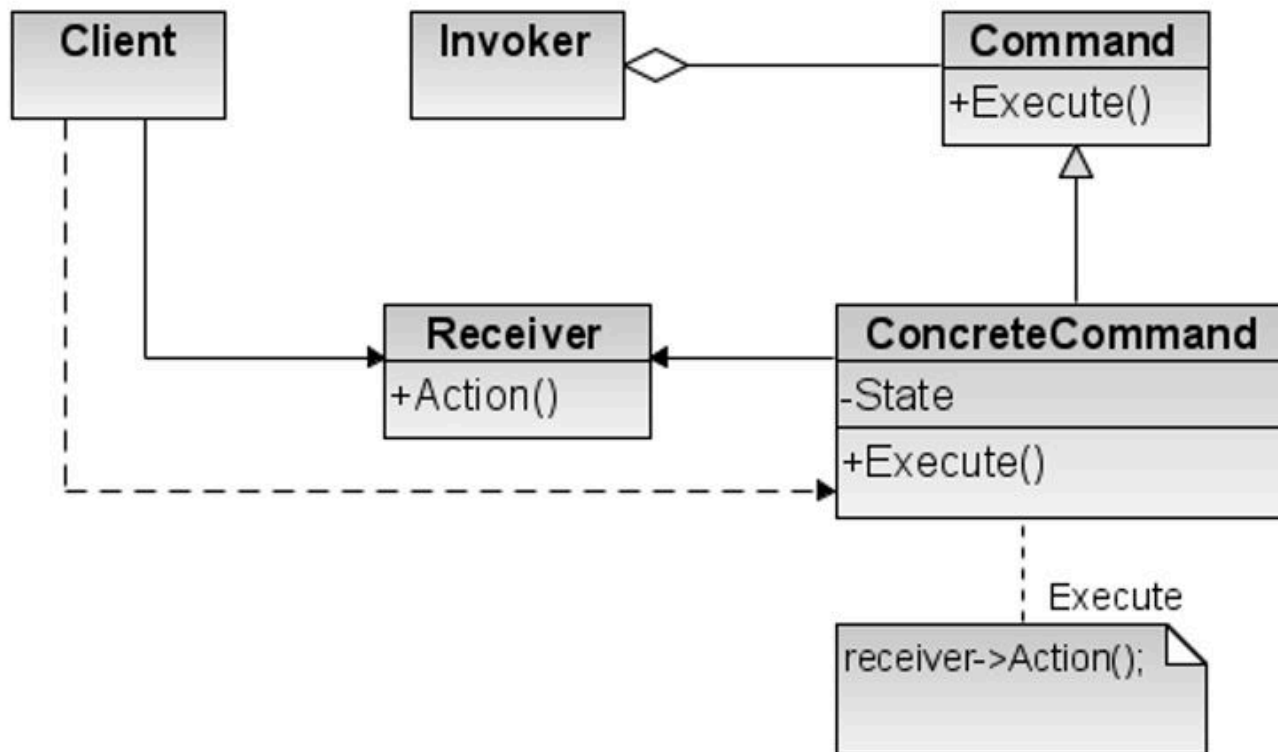
===

No desenvolvimento de aplicações existem muitas situações em que se tem que se lidar com a gestão da execução de comandos (ordens ou operações):

- adiar a execução de operações
- armazenamento do histórico de operações,
- definição de macros (definir uma sequencia de operações que poderá ser executada várias vezes)
- associar eventos do utilizador a execução de operações.
- desfazer operações realizadas

Command | Solução

O padrão comando permite encapsular um comando ou pedido num objecto



Command | Participantes

- **Command** - declara uma interface (ou classe abstrata) para a execução de uma operação
- **ConcreteCommand** - define uma ligação entre um objeto recetor e uma ação. Implementa a operação execute invocando a operação correspondente (s) no Recetor
- **Client** - cria um objeto ConcreteCommand e define seu recetor
- **Invoker** - pede o comando para executar o pedido
- **Receiver** - sabe como realizar as operações (Action) relacionadas com a realização do pedido.

Command | Interface Command

Declara uma interface (ou classe abstrata) para a execução de uma operação

```
public interface Command {  
    void execute() ;  
    void unExecute(); //opcional  
}
```

Command | Receiver

sabe como realizar as operações (action) relacionadas com a realização do pedido.

```
public class Receiver {  
    void action1() {  
        ...  
    }  
    void action2(){  
        ...  
    }  
}
```


Command | Concrete Command

Define uma ligação entre um objeto recetor e uma ação. Implementa a operação `execute` (e às vezes a de `unExecute`) invocando a operação correspondente (s) no Recetor

```
public ConcreteCommand implements Command{
    private Receiver receiver;

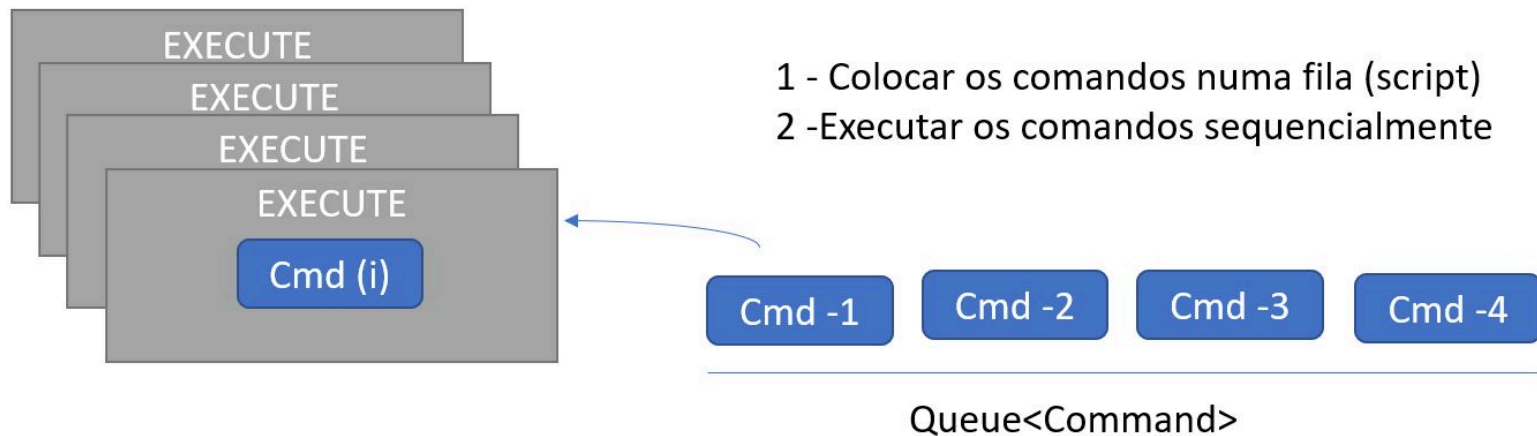
    public ConcreteCommand(Receiver receiver)    {
        this.receiver=receiver;
    }

    public void execute(){
        this.receiver.action1();
    }

    public void unExecute(){
        this.receiver.action2();
    }
}
```

Command | Fila de Commands

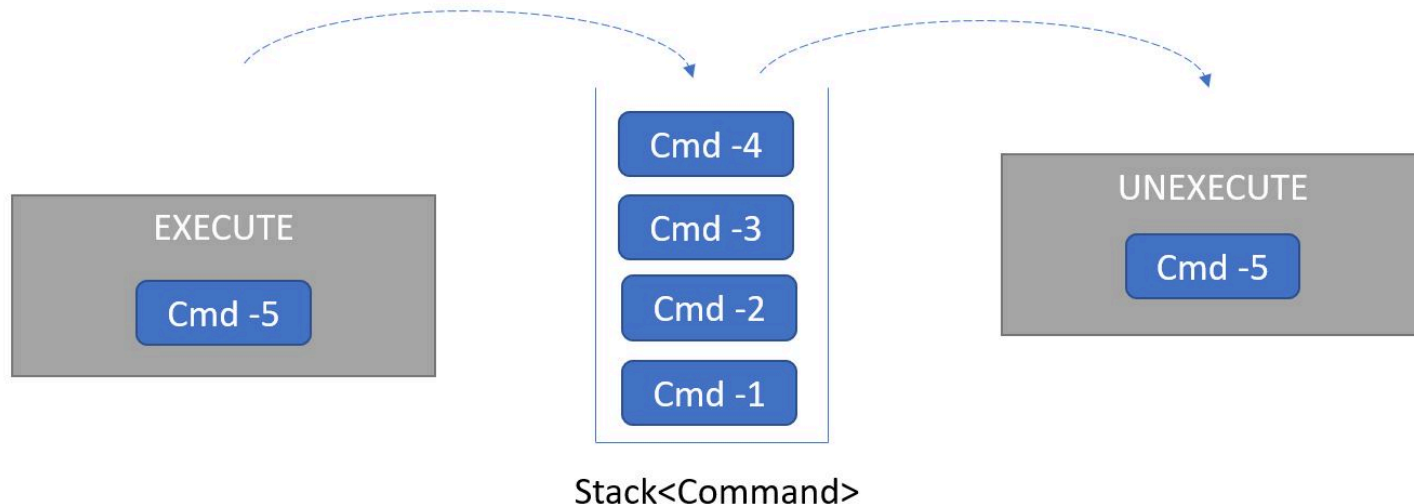
Os comandos podem ser manipulados de forma a colocá-los em filas de espera e atrasar a sua execução.



Command | Stack de Commandos

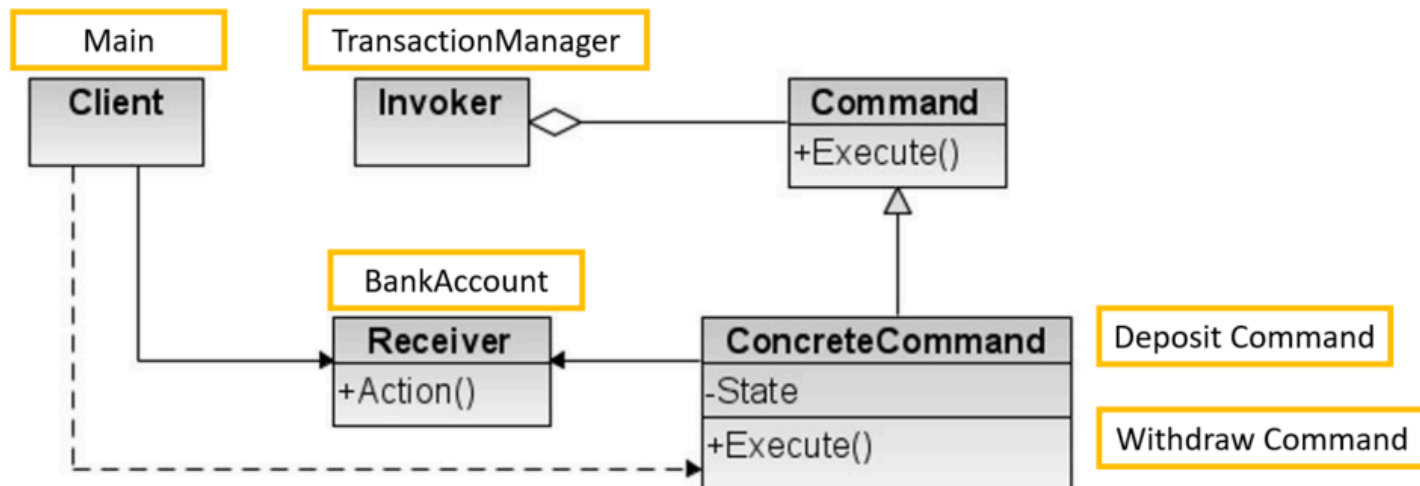
Os comandos podem ser manipulados para suportar a execução da inversão das acções das operações relativas a um comando (undo).

- 1 – Cada vez que um comando é executado é guardado numa pilha
- 2 – Quando se pretende fazer, undo retira-se o comando da pilha e faz-se unExecute



Command | Exemplo de Aplicação

- O seguinte repositório contém a aplicação do padrão **command**.
https://github.com/estsetubal-pa/Command_BankAccount.git
- O programa exemplifica como usar o padrão **command** para gerir as operações sobre uma Conta Bancária (depósitos e levantamentos).
- Permite obter o histórico de transações e "desfazer"(*undo*) uma transação.



Command | Receiver

```
public class BankAccount {  
    private String accountHolder;  
    private double balance;  
  
    public void deposit(double amount) {  
        balance += amount;  
    }  
  
    public void withdraw(double amount) {  
        if (balance >= amount) {  
            balance -= amount;  
        }  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}
```

Command | Concrete Command

```
public class DepositCommand implements Command {
    private BankAccount account;
    private double amount;

    public DepositCommand(BankAccount account, double amount) {
        this.account = account;
        this.amount = amount;
    }
    @Override
    public void execute() {
        account.deposit(amount);
    }
    @Override
    public void unExecute() {
        account.withdraw(amount);
    }
}
```

Command | Concrete Command

```
public class WithdrawCommand implements Command {  
    private BankAccount account;  
    private double amount;  
  
    public WithdrawCommand(BankAccount account, double amount) {  
        this.account = account;  
        this.amount = amount;  
    }  
  
    @Override  
    public void execute() {  
        account.withdraw(amount);  
    }  
  
    @Override  
    public void unExecute() {  
        account.deposit(amount);  
    }  
}
```

Command | Invoker

```
public class TransactionManager {
    private List<Command> transactionHistory = new ArrayList<>();

    public void executeCommand(Command command) {
        command.execute();
        transactionHistory.add(command);
    }

    public String showTransactionHistory() {
        String str="Transaction History:\n";
        for (Command command : transactionHistory) {
            str+="- " + command.getClass().getSimpleName()+"\n";
        }
        return str;
    }

    public void undo() {
        Command cmd= this.transactionHistory.remove(transactionHistory.size()-1);
        cmd.unExecute();
    }
}
```


Command | Client

```
public class Main {  
    public static void main(String[] args) {  
        BankAccount account = new BankAccount("John Doe", 1000.0);  
        Command depositCommand = new DepositCommand(account, 500.0);  
        Command withdrawCommand = new WithdrawCommand(account, 200.0);  
        TransactionManager transactionManager = new TransactionManager();  
        // Execute commands  
        transactionManager.executeCommand(depositCommand);  
        transactionManager.executeCommand(withdrawCommand);  
        // Show transaction history  
        System.out.println(transactionManager.showTransactionHistory());  
        // Check final balance  
        System.out.println("Final balance: $" + account.getBalance());  
        // Undo  
        transactionManager.undo();  
        // Show transaction history after undo  
        System.out.println(transactionManager.showTransactionHistory());  
        // Check final balance after undo  
        System.out.println("Final balance: $" + account.getBalance());  
    }  
}
```

Command | Pros e Contras

Prós

- Princípio da Responsabilidade Única. É possível dissociar as classes que invocam operações das classes que realizam estas operações.
- Princípio aberto/fechado. É possível introduzir novos comandos na aplicação sem quebrar o código de cliente existente.
- É possível implementar desfazer/redo.
- Pode-se implementar a execução diferida de operações.
- É possível montar um conjunto de comandos simples num complexo.

Contra

- O código pode tornar-se mais complicado, uma vez que se está a introduzir uma camada extra entre remetentes (invokers) e receptores (receivers).

Command | Treino em casa 🦵 💻

Complete o exercício proposto em:

https://github.com/estsetubal-pa/Command_BankAccount.git

Command | Bibliografia

- <https://refactoring.guru/design-patterns/command>
- <https://youtu.be/UfGD60BYzPM>